

Rappresentazione dell'informazione

Di cosa parleremo:

01 Informazione e informatica

03 Sistemi posizionali

05 Convertire tra basi

07 Numeri negativi

09 Rappresentazione del testo

02 Bit, byte e configurazioni

04 Contare in basi diverse

06 Rappresentabilità e overflow

08 Numeri reali (floating point)

01

Informazione e informatica

Che cosa elaboriamo, e in che forma.

Informatica

Informat(ion) + (automat)ique: il trattamento automatico dell'informazione.

Disciplina scientifica che si occupa dello studio dell'elaborazione delle informazioni.

Si dice **Informazione** l'insieme di dati, correlati tra loro, da cui un'idea o un fatto prende forma ed è comunicata.

Analogico contro digitale

Analogico

Valori in stretta *analogia* col fenomeno, tipicamente **continui** (infiniti valori)



Digitale

Informazione codificata come sequenza di simboli da un insieme **discreto** e finito.



Il calcolatore gestisce l'informazione solo in forma digitale

Come si trasforma qualsiasi cosa in una sequenza di simboli discreti che una macchina può memorizzare?

02

Bit, byte e configurazioni

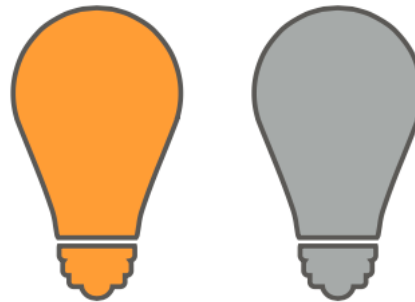
Il bit: un valore fra due

Un **bit** (*binary digit*) vale 0 oppure 1.

Fisicamente: uno stato con due sole possibilità: tensione bassa o alta, interruttore spento o acceso.

Perché due soli simboli?

Il computer è **fisicamente** fatto di transistor, che distinguono in modo affidabile solo *due* stati.



1

0

Quante configurazioni può assumere una sequenza di 2 bit?

A	B
0	0
0	1
1	0
1	1

2 bit → 4 combinazioni

In generale, una sequenza di k bit può assumere 2^k combinazioni.

Dal bit alla doubleword

NOME	BIT	CONFIGURAZIONI
bit	1	2
nybble	4	16
byte	8	256
halfword	16	65 536
word	32	≈ 4,3 miliardi
doubleword	64	≈ 1,8 × 10 ¹⁹

Il **byte** (8 bit, 256 valori) è l'unità pratica: standard universale dal 1964, grande quanto basta per un carattere.



Bit

0 o 1, 2 valori rappresentabili



Byte

8 bit, 256 valori rappresentabili

Kilobyte (KB)

1024 byte

Nota Bene

A seconda dell'ambito a volte si considerano KB, MB, etc. come 1000 byte, un milione di byte, etc.

Megabyte (MB)

1024 KB

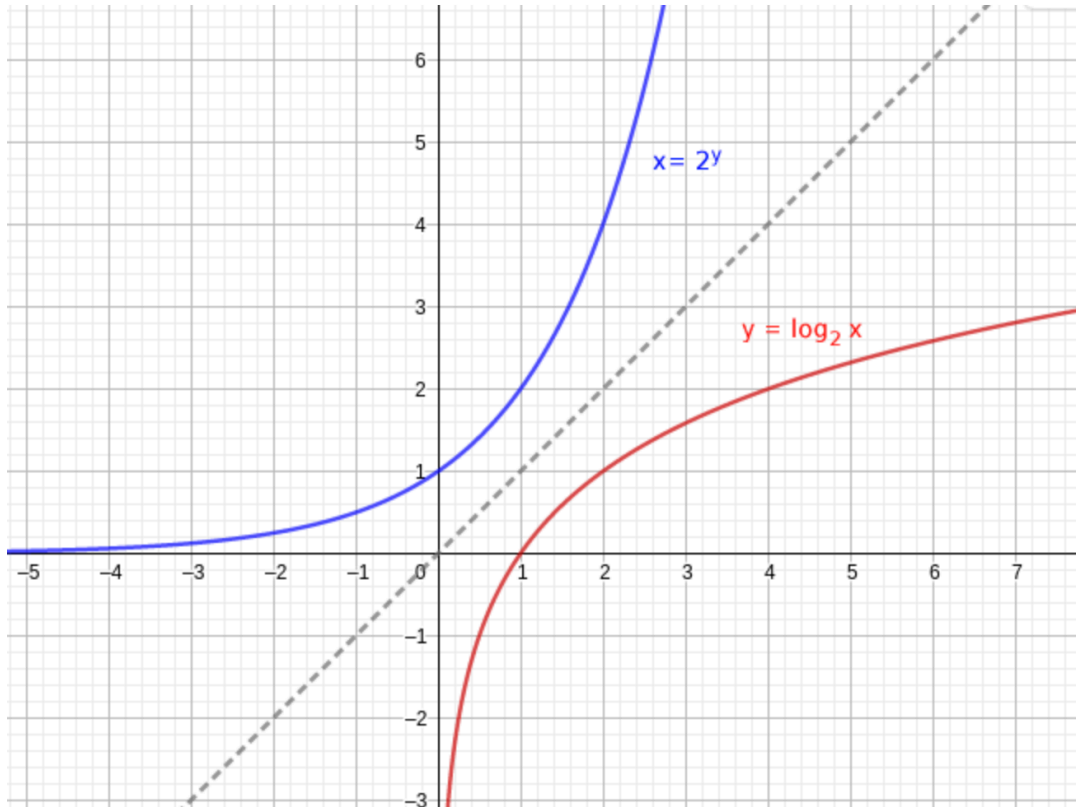
Gigabyte (GB)

1024 MB

Terabyte (TB)

1024 GB

Quale è il numero di bit necessario per rappresentare 100 valori diversi?



Funzione esponenziale:

<https://www.matematgeogebra.it/page-esponenziale/code-29/>

Funzione logaritmica:

<https://www.matematgeogebra.it/page-esponenziale/code-30/>

03

Sistemi di numerazione posizionali

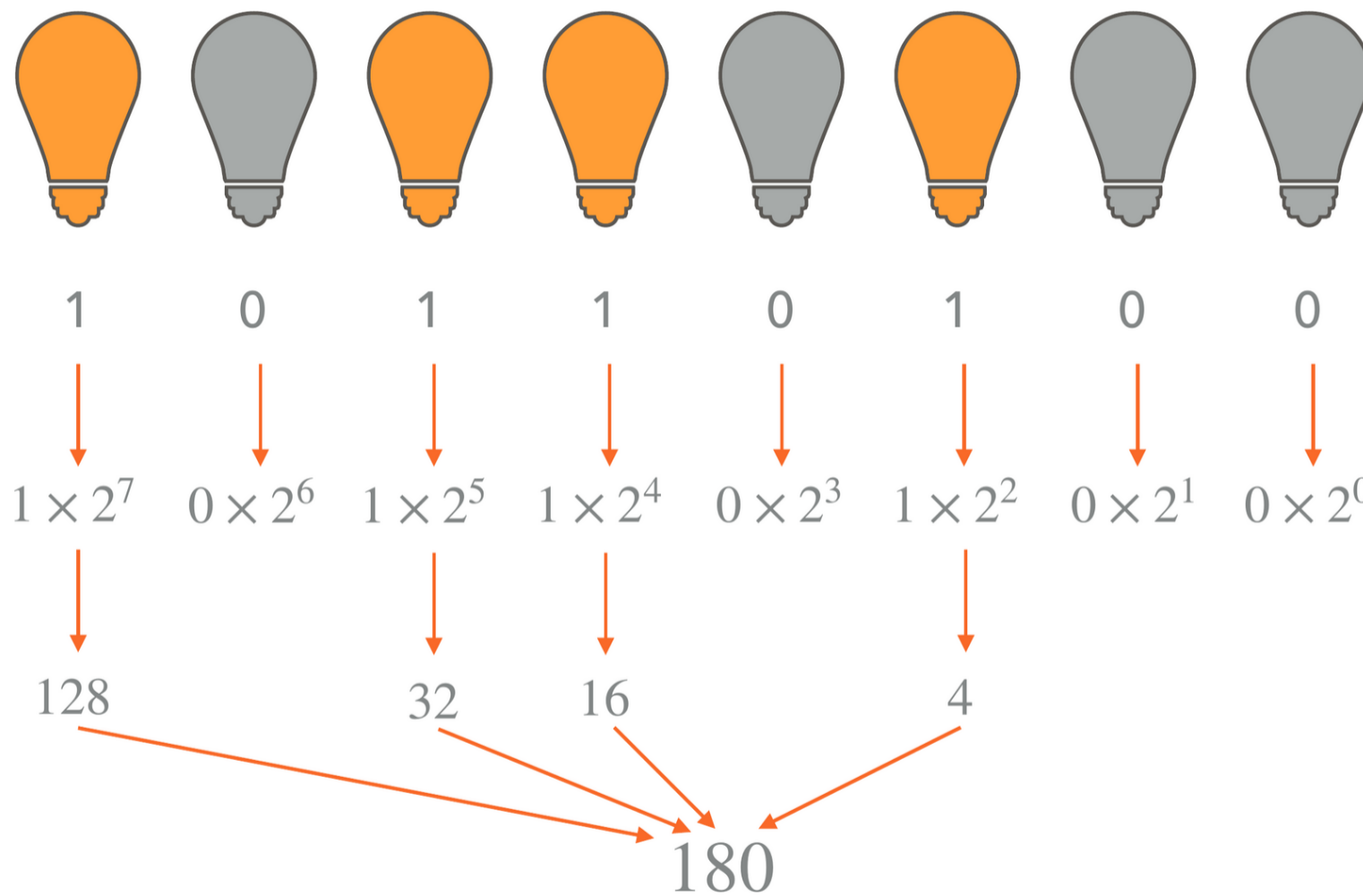
Il valore di una cifra dipende dalla posizione

Ogni posizione vale una potenza della base, crescente da destra verso sinistra.

3	4	2	9
↓	↓	↓	↓
$\times 10^3$	$\times 10^2$	$\times 10^1$	$\times 10^0$
3000	400	20	9
$3000 + 400 + 20 + 9 = 3429$			

Ogni bit è una lampadina

10110100 = 180



Rappresentiamo il numero **110101** in codifica decimale

Perché base 10? E la regola generale

Perché dieci cifre?

Nessun motivo profondo: abbiamo dieci **dita**. Nessun vantaggio matematico: si può contare con basi diverse.



La regola, per ogni base r

$$N = \sum d_i \cdot r^i$$

I numeri romani non sono posizionali

1 - I	10 - X	100 - C
2 - II	20 - XX	200 - CC
3 - III	30 - XXX	300 - CCC
4 - IV	40 - XL	400 - CD
5 - V	50 - L	500 - D
6 - VI	60 - LX	600 - DC
7 - VII	70 - LXX	700 - DCC
8 - VIII	80 - LXXX	800 - DCCC
9 - IX	90 - XC	900 - CM
1000 - M	2000 - MM	3000 - MMM

04

Contare in basi diverse

Quando le cifre non bastano

SISTEMA NUMERICO DECIMALE	SISTEMA NUMERICO BINARIO	SISTEMA NUMERICO ESADECIMALE
0	0	0
1	1	1
2		2
3		3
4		4
5		5
6		6
7		7
8		8
9		9
		A
		B
		C
		D
		E
		F

Base 16 (esadecimale)

Servono più di dieci cifre: si prendono in prestito le lettere.

A = 10, B = 11, ... F = 15.

$$21_{16} = 1 + 2 \cdot 16 = 33_{10}$$

$$10_{16} = 0 + 1 \cdot 16 = 16_{10}$$

$$10_2 = 0 + 1 \cdot 2 = 2_{10}$$

$$\begin{aligned} C2A5_{16} &= 5 + A \cdot 16 + 2 \cdot 16 \cdot 16 + C \cdot 16 \cdot 16 \cdot 16 = 5 + \\ &10 \cdot 16 + 2 \cdot 64 + 12 \cdot 4096 = 49829_{10} \end{aligned}$$

04 • Applichiamo la regola

Quanto vale 7A5 in base 16?

$$\begin{aligned} 7A5_{16} &= 7 \times 16^2 + 10 \times 16^1 + 5 \times 16^0 \\ &= 1792 + 160 + 5 = 1957_{10} \end{aligned}$$

Quanto vale 65 in base 16?

E in base 7?

E in base 8?

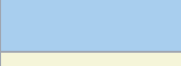
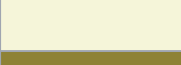
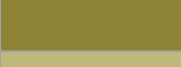
Sistema ottale

E in base 5?

(Domanda trabocchetto)

Decimale, binario, esadecimale a confronto

DECIMALE	BINARIO	ESADECIMALE
0	0000	0
5	0101	5
9	1001	9
10	1010	A
13	1101	D
15	1111	F

Nome ⇅	Esempio ⇅	Esadecimale ⇅	RGB		
			R ⇅	G ⇅	B ⇅
Acquamarina		#7FFFD4	127	255	212
Albicocca		#FBCEB1	251	206	177
Amaranto		#E52B50	229	43	80
Ambra		#FFBF00	255	191	0
Ametista		#9966CC	153	102	204
Anguria		#FC6C85	252	108	133
Antracite		#293133	41	49	51
Aragosta		#ED7465	237	116	101
Arancione		#FFA500	255	165	0
Ardesia		#708090	112	128	144
Argento		#C0C0C0	192	192	192
Asparago		#87A96B	135	169	107
Avio		#5D8AA8	93	138	168
Avorio		#FFFFF0	255	255	240
Azalea		#D3305D	211	48	93
Azzurro		#007FFF	0	127	255
Azzurro fiordaliso		#ABCDEF	171	205	239
Beige		#F5F5DC	245	245	220
Beige-oliva chiaro		#908435	144	132	53
Beige verdastro		#BEBD7F	190	189	127

RGB: (Red, Green, Blue)

HEX: #[RR][GG][BB]

05

Convertire tra basi

05 · Da una base a decimale

Si applica la regola generale

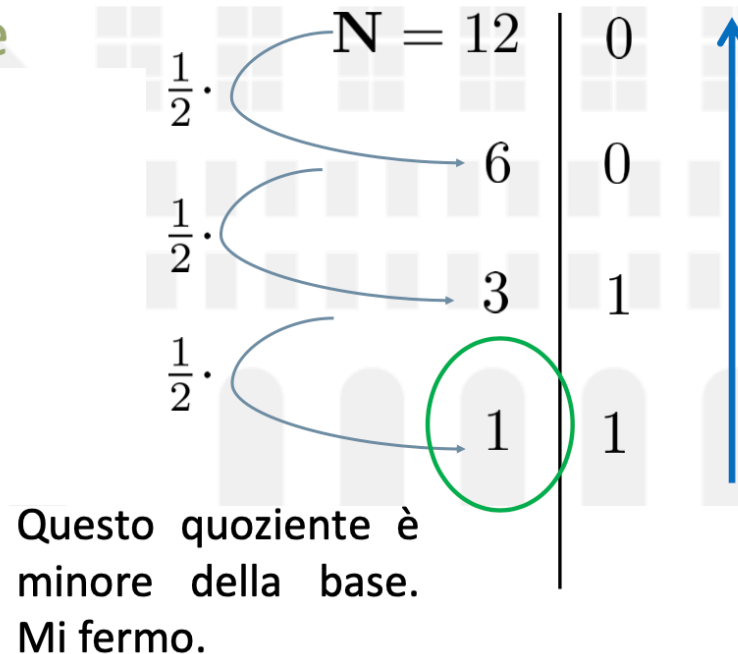
$$\begin{aligned} 1101_2 &= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 8 + 4 + 0 + 1 = 13_{10} \end{aligned}$$

Valutazione di un polinomio

Da decimale a qualsiasi altra base

1. **Dividere** il valore numerico **N** per la base **r** fino a quando l'ultimo quoziente è minore della base stessa. In questo processo, tenere traccia di tutti i quozienti e tutti i resti.
2. «**Leggere**», in ordine inverso, tutti i resti delle divisioni. **Scrivere** tutti i valori da sinistra verso destra.

Esempio da **decimale**
a **binario**:

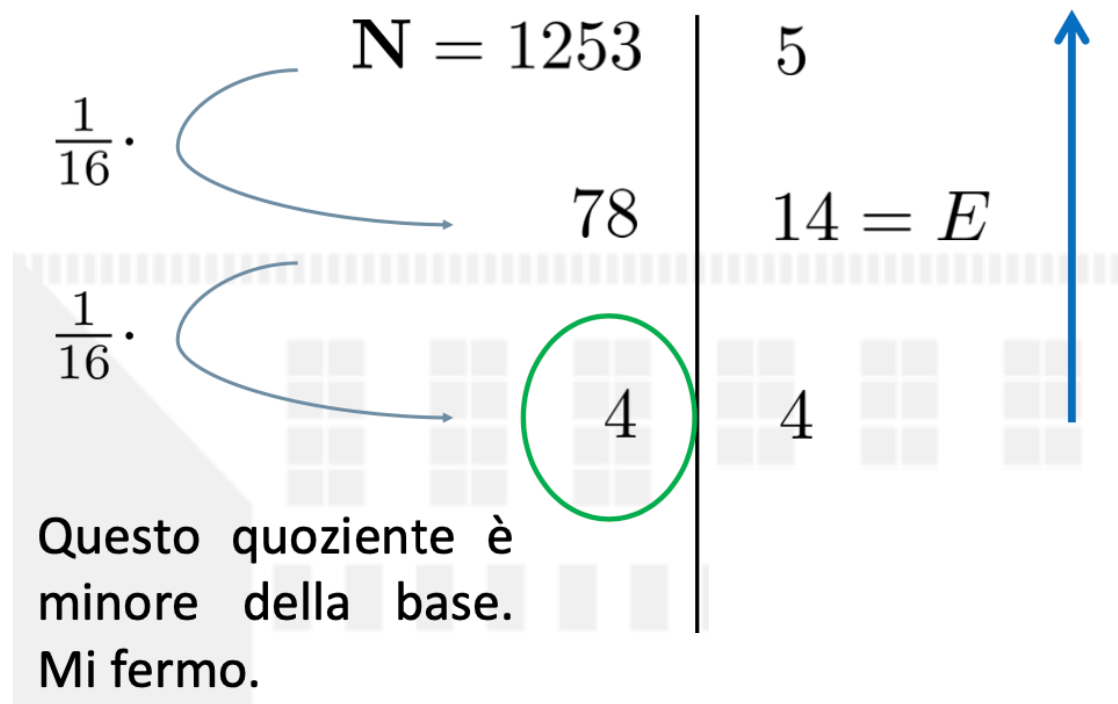


$$N = (12)_{10} = (1100)_2$$

Provate a scrivere la vostra data di nascita (giorno e mese) in codice binario

GG / MM

Esempio da **decimale** a **esadecimale**:



$$N = (1253)_{10} = (4E5)_{16}$$

**Provate a scrivere la vostra data di nascita (giorno, mese e anno) in
codice esadecimale**

GG / MM / AAAA

05 • La scorciatoia binario ↔ hex

$16 = 2^4 \rightarrow$ una cifra hex = 4 bit

Basta raggruppare i bit a quattro a quattro, da destra.

0111	1111	0001	1010
↓	↓	↓	↓
7	F	1	A

$0111111100011010_2 = 7F1A_{16}$

L'esadecimale è compatto e comodo

06

Rappresentabilità e overflow

Con un numero finito di bit si rappresenta solo un intervallo finito di valori.

06 · L'intervallo rappresentabile con n bit

Con n bit → da 0 a $2^n - 1$



$$000 = 0$$



$$111 = 7$$

3 bit → $2^3 = 8$ valori, da 0 a 7.

Posso scrivere 41 con 5 bit?

41 in binario è 101001: servono 6 bit. Con 5 bit il massimo è 11111 = 31. Quindi no: va in **overflow**

Può accadere anche come risultato di operazioni su numeri già rappresentati (e.g. somme e sottrazioni)!

In pratica · i colori RGB

Tre canali (R, G, B). A 8 bit per canale → $256^3 \approx 16,7$ milioni di colori. Più bit spendiamo, più colori: la qualità è una questione di bit.

07

Numeri negativi

07 · Idea 1 · il bit di segno

Rappresentazione sign-and-magnitude: il primo bit dice il segno



10001110 = -14 · 00011001 = +25

Il problema

Esistono due zeri: 00000000 = +0 e 10000000 = -0

07 · Idea 2 · il complemento a due

Il primo bit vale una potenza negativa



Numero (7 bit)

Bit interpretato "con segno meno"
(i.e., se a 1 vale -128)

10001110 equivale a $-10000000 + 00001110$

ovvero $-128 + 8 + 4 + 2 = -128 + 14 = -112$

In generale:

Con n bit il primo bit rappresenta il valore $-2^{(n-1)}$
per gli altri non cambia nulla

In complemento a due, su 8 bit...

quali valori rappresentano `11111111` e `01111111`?

quale è l'intervallo di valori rappresentabili?

08

Numeri reali (floating point)

Rappresentare i numeri con la virgola.

08 · Lo standard IEEE 754

Segno · esponente · mantissa

valore = $\pm$ mantissa $\times 2^{\text{esponente}}$ — single precision, 32 bit



In *double* (64 bit): 1 + 11 + 52. Previsti valori speciali: NaN, $+\infty$, $-\infty$.

Argomento complesso: corso di *Architettura degli Elaboratori*

08 · I tranelli dei float

I bit sono finiti → i calcoli sono approssimati

Le operazioni **non** sono necessariamente associative.

Non testare l'uguaglianza esatta fra due float: verifica se sono "abbastanza vicini".

Operazioni matematicamente equivalenti possono dare risultati diversi.

La disciplina che studia risultati affidabili da calcoli approssimati è il **calcolo scientifico**.

09

Rappresentazione del testo

Anche le lettere devono esser rappresentate numericamente.

Senza codifica, sono solo bit

```
01001101 01100001  
01110010 01101001  
01100001 00111111
```

Una **codifica** (encoding) è una tabella che associa a ogni carattere un numero. È l'encoding a dare significato alla sequenza.

7 bit, 128 caratteri

ASCII (anni '60) rappresenta lettere latine, cifre, punteggiatura e spazio con 7 bit.

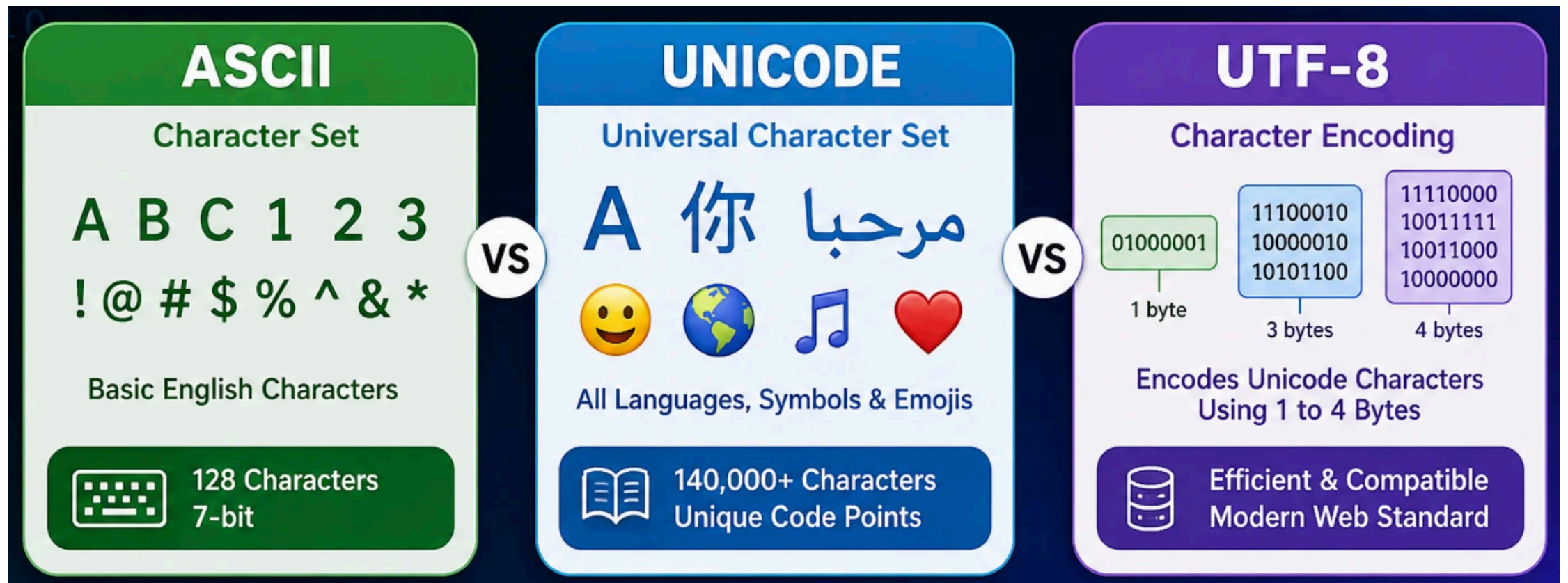
CARATTERE	CODICE
' '	32
'0'	48
'A'	65
'a'	97

Numero ≠ carattere

Il carattere '0' ha codice 48; la lettera 'A' ha codice 65. Il glifo che *disegna* lo zero è una cosa, il *numero* zero un'altra.

128 caratteri bastano per l'inglese, non per il mondo

Unicode assegna un numero a ogni carattere di ogni lingua (ed emoji). UTF-8 resta compatibile con ASCII per i primi 128 caratteri, ma usa fino a 4 byte per gli altri. Per questo le lettere accentate e gli ideogrammi occupano più byte.



01110011011110110010101000100011110011001100000010010001001110
11101011100100000110111101010001100110010010000001100011000011
111010001011101101111111111011000110000001111001110010010110000
01111110111001101010111010000010011010011100011000100111011101
101010111001000111111111101100001111000101010001111111001101000
00111011010010111011000011100100000110111010111001110000001000
00111110001011100001111011011111111000111100001101010011111100
110110

In sintesi

Tutto è una sequenza di bit;
la *codifica* decide cosa significa